

Token Economics and Consumption Control

OpenClaw precision-strike strategies in practice · Three lines of defense: compression, routing, circuit-break

PROMPT COMPRESSION

Structured extraction · Less redundancy

ON-DEMAND ROUTING

Multi-Endpoint · Token diet

MONITORING & BREAK

Threshold alerts · Auto-rollback

Today's Agenda

From background to deployment — four chapters to master Token cost governance

01

Background & Challenges

Real-world out-of-control Token scenarios and cost structure

**02**

Three Precision-Strike Strategies

Prompt compression · On-demand routing · Monitor & break

**03**

OpenClaw Engineering Practice

Code-level Token meter and multi-Endpoint dispatching

**04**

Results Review & Action Checklist

ROI data and 5 key steps for team adoption



Why are Tokens the new "utilities"?

In the LLM era, every inference call is a precisely metered charge

73%

AI project cost

Share of total AI app cost from Token call fees

5x

Call volume growth

Doubling rate of enterprise LLM API calls over the last 12 months

42%

Optimizable redundancy

Share of invalid or duplicated context in a typical Prompt

Source: OpenClaw internal baseline + industry average (n=120 projects)

Three Typical Out-of-Control Scenarios

The three most common Token bill assassins at quarterly reconciliation



PROMPT BLOAT

Context Bloat

Users stuff entire education backgrounds and full PDFs into every Prompt — single requests easily exceed 8K Tokens.



OVER-MODEL

Model Mismatch

Even simple if-else logic calls GPT-4-class models — 30× the price of equivalent small models.



RUNAWAY LOOP

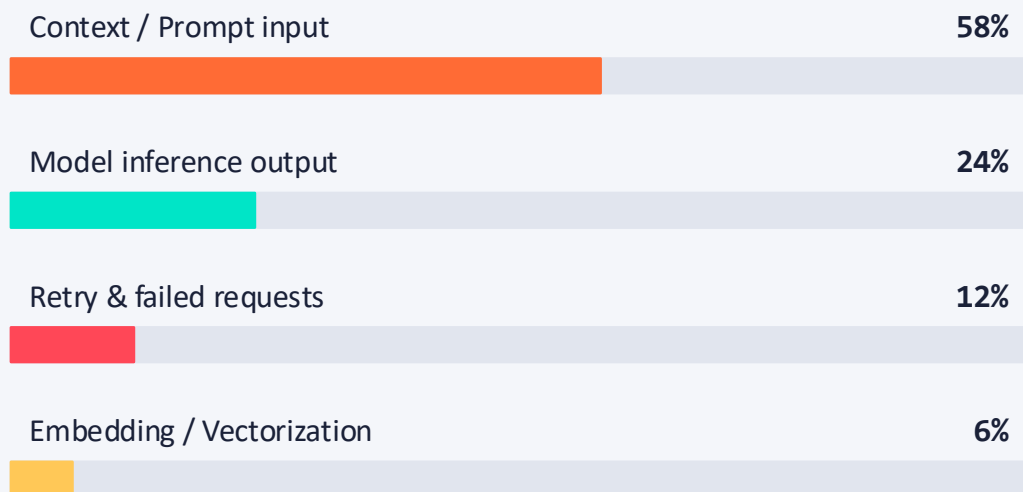
Failure Retry

Exception branches retry infinitely without circuit-breakers — one bug burns a full day's budget in 30 minutes.

How a Token bill is calculated

Decompose one agent call to see where every cent goes

Typical call cost composition



INSIGHT

\$ 0.47

Average cost per agent task

~70% is compressible input context and wasted retries — the attack surface for OpenClaw's three strategies.

OpenClaw's Precision-Strike Philosophy

No crude rate-limiting — instead, build observable, intervenable, rollback-able guardrails at every layer

“

*Tokens aren't a limit — they're a budget. Treat every call as a conscious expense,
not a free default operation.*

– OpenClaw Engineering Principles

01

Observable

Every Token must be metered and traceable

02

Intervenable

Inject strategy and compression logic before and after each call

03

Rollback-able

Auto-break on per-task overrun — no downstream contamination

Three Precision-Strike Strategies

From input to dispatch to runtime — build a multi-layered Token defense

INPUT LAYER

STRATEGY 01

Prompt Compression

Compression

Use Copilot to structure-extract verbose background materials, cutting context redundancy.

DISPATCH LAYER

STRATEGY 02

ON-DEMAND ROUTING

Routing

OpenClaw configures multiple Model Endpoints — simple logic goes to small models for a "Token diet".

RUNTIME LAYER

STRATEGY 03

Monitor & Break

Circuit-Break

A code-level Token meter triggers automatic rollback when a single task exceeds threshold.

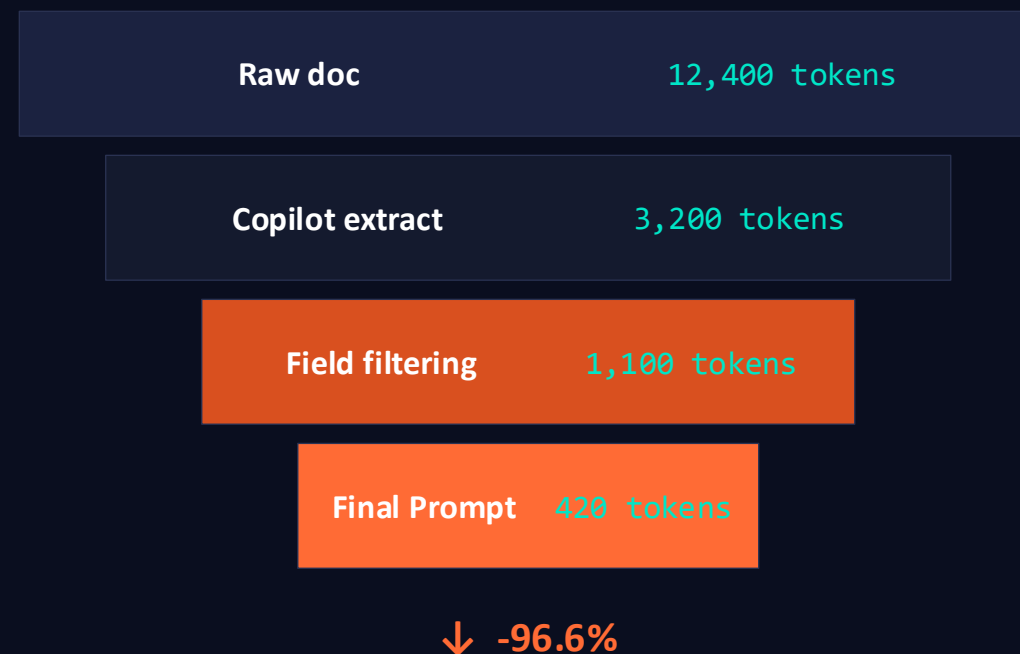
Strategy 1 · Prompt Compression

Copilot compresses verbose background into structured fragments — cut invalid Tokens at the source

Core actions

- Identify redundancy — long-tail text like education history, product manuals, contract templates
- Structured extraction — Copilot → JSON/fields, drops natural-language fluff
- Dynamic injection — Prompt only carries the fields the task actually needs
- Cache reuse — structured results for the same entity reused within 24h

Compression funnel



Copilot Structured Extraction Workflow

5 steps to turn unstructured material into Prompt-ready fields



```
// Copilot extract schema
```

```
{ "name": str, "degree": str, "school": str, "major": str, "year": int, "achievements": str[3] }
```

- Keep only the 6 task-relevant fields — remove 90% natural-language fluff
- Schema-ized fields are reusable across tasks, avoiding redundant extraction

Case Study · Education Background Compression

Recruiting scenario: real comparison from 2,800 → 180 tokens for a candidate resume

BEFORE · Raw Prompt

2,800 tokens

"Zhang San enrolled in Tsinghua University's Department of Computer Science and Technology in September 2018, majoring in AI. He received the National Scholarship and won an Asia Regional Bronze in the ACM contest... (followed by 2,500 words of background narrative, club experience, award details, recommendation letter excerpts)"

AFTER · After Copilot extraction 180 tokens

```
{
  "name": "Zhang San",
  "degree": "Bachelor",
  "school": "Tsinghua University",
  "major": "Computer Science · AI track",
  "year": 2022,
  "achievements": [
    "National Scholarship",
    "ACM Asia Regional Bronze"
  ]
}
```

Real Results of Strategy 1

12 production workflows over 2 weeks, all switched to Copilot structured injection

AVERAGE PROMPT REDUCTION

-78%

Average Prompt size reduction across 12 workflows

Largest single-flow reduction: 96.6%

Monthly Token bill

↓ 64%

Average response latency

↓ 41%

Cache hit rate

↑ 3.2×

Live first-response time (P95)

↓ 38%

Strategy 2 · On-Demand Routing

Multi-Endpoint routing inside OpenClaw — "don't use a butcher's cleaver to kill a chicken"

Why a Token diet

Cost-sensitive

GPT-4 vs 7B small model: 30× price gap; simple classification doesn't need high-end models

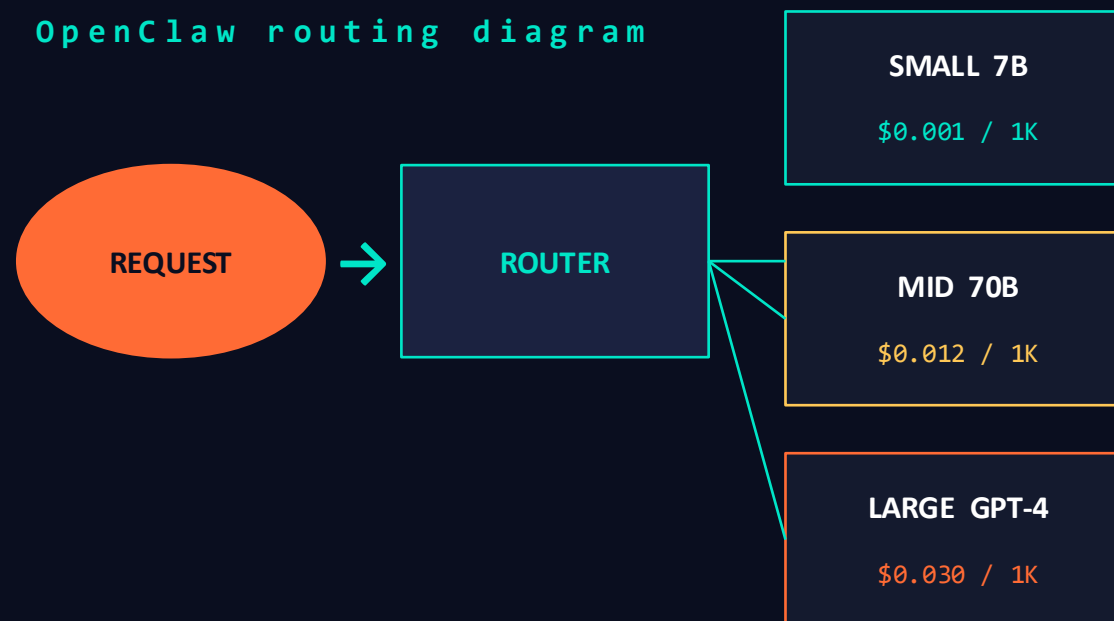
Latency-sensitive

Local small models respond in 200ms; cloud large models average 1.8 seconds

Controllability

Structured extraction, keyword matching, rule-based decisions can all run on small models

OpenClaw routing diagram



Auto-route by complexity score

Model Endpoint Configuration Matrix

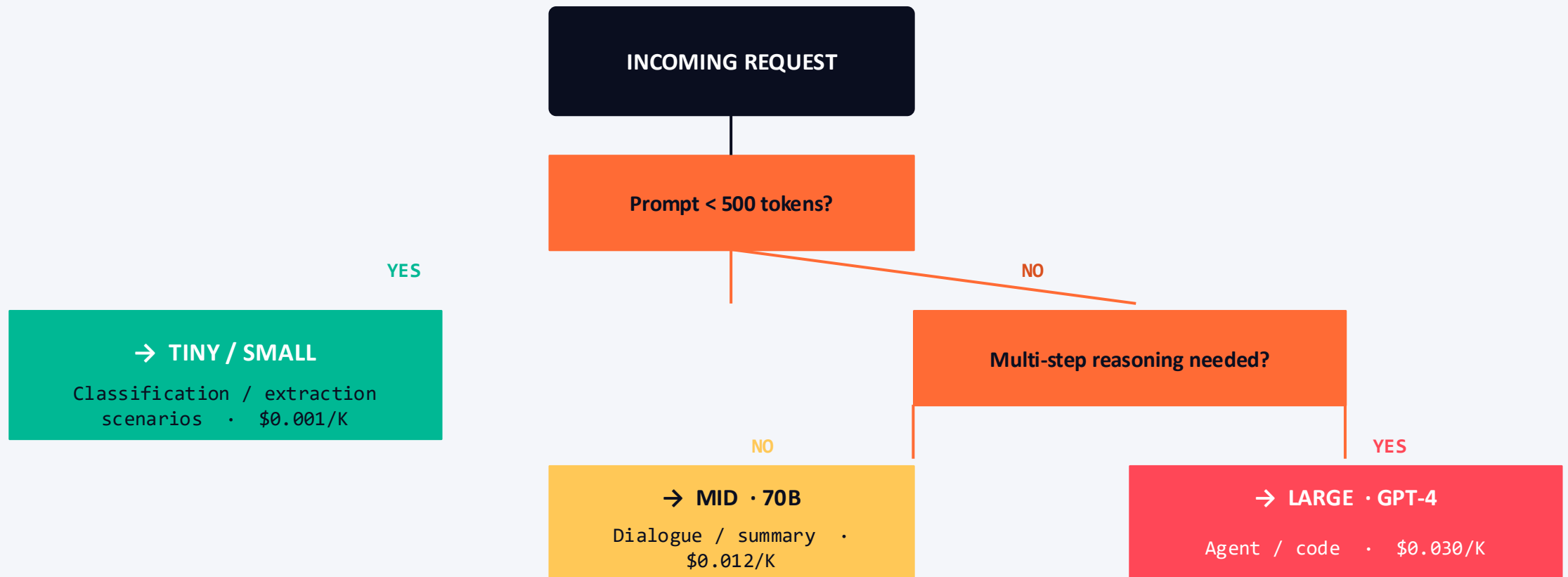
OpenClaw splits Endpoints into 4 tiers by task complexity — transparent pricing

Task type	Recommended Endpoint	Example	Price / 1K Token	Budget share
Classification / routing decisions	Tiny · 1.5B	Intent recognition, profanity filtering	\$0.0004	5%
Structured extraction	Small · 7B	Resume fields, contract clauses	\$0.0010	20%
Dialogue & summary	Mid · 70B	Customer support dialogue, document summary	\$0.0120	45%
Complex reasoning	Large · GPT-4 tier	Multi-step code generation, Agent decisions	\$0.0300	30%

Rule: every Workflow must explicitly declare `endpoint_tag` — undeclared defaults to Tiny → forces developers to actively pick a tier.

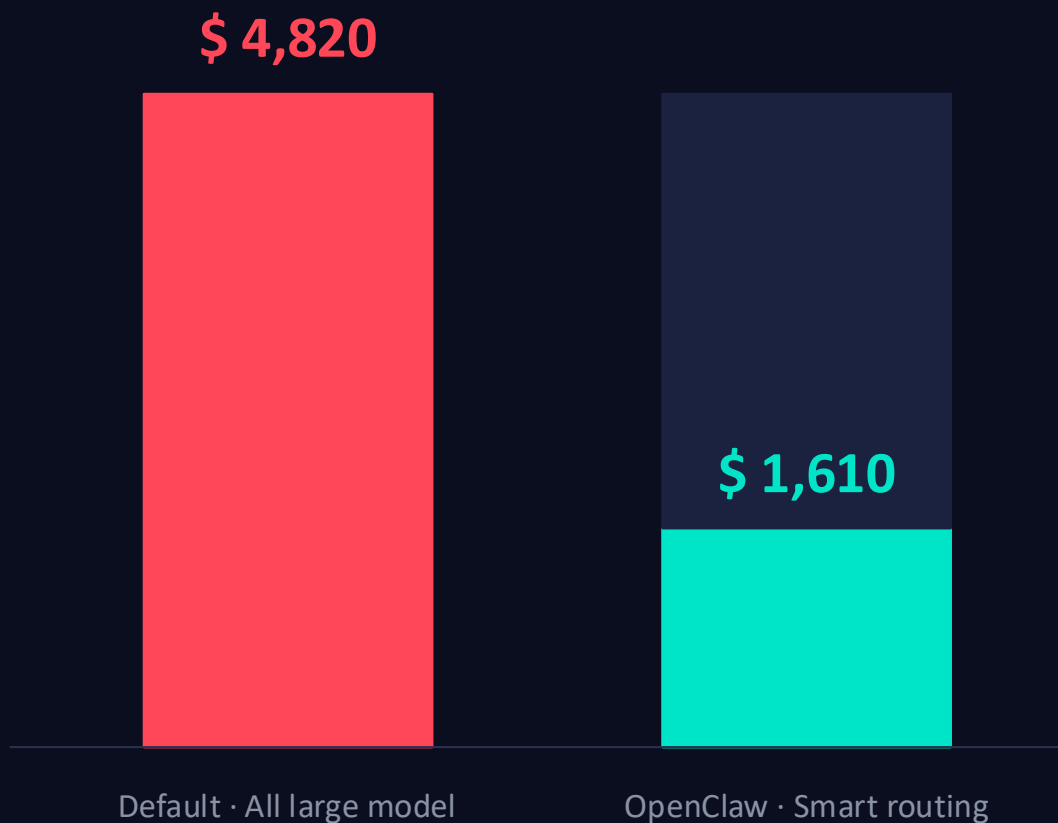
Token Diet Decision Tree

A 30-ms pre-call decision determines how many "calories" this request gets



Cost Curve from Token Diet

Same workflow: default all-large-model vs OpenClaw smart routing



MONTHLY SAVING

-66.6%

Monthly bill drops from \$4,820 to \$1,610

- 62% traffic routed to Tiny / Small tiers
- Only 8% complex tasks stay on GPT-4 tier
- P95 latency drops 41% simultaneously

Strategy 3 · Monitor & Break

Code-level Token meter · Threshold alerts · Auto-rollback

LIVE TOKEN METER

Current task usage

8,420 / 10,000



80% threshold

Status **WARN · Approaching threshold**

Action **Downgrade to Mid tier + abort downstream inference**

Rollback **Checkpoint @step3 in place**

Three layers of circuit-break action

Throttle · Throttle

After 60% threshold, lower concurrency and tighten gradually

Downgrade · Downgrade

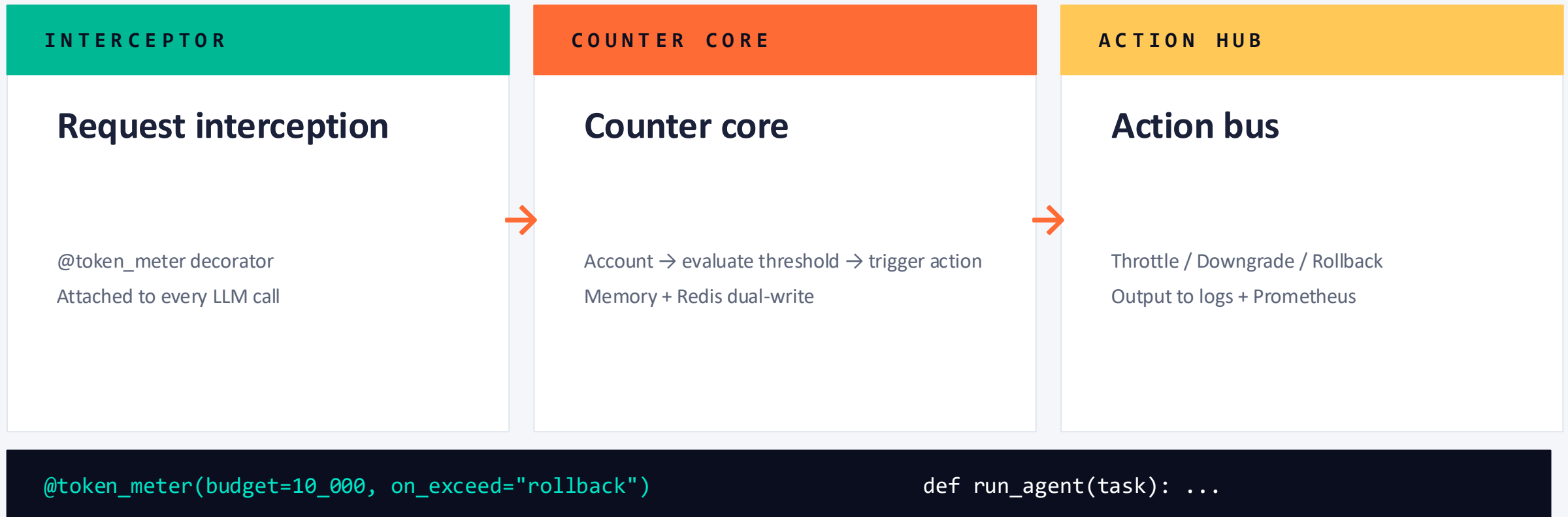
After 80% threshold, force-route to smaller model

Rollback · Rollback

After 100% threshold, abort task and replay snapshot

Token Meter Architecture

Non-intrusive code-level integration: decorator + middleware + sidecar metrics



Threshold Trigger & Rollback Timeline

From counting to rollback, millisecond response prevents a single task from "burning through" the budget



After circuit-break triggers: this task is excluded from user-visible results, but the full event chain is written to Trace for post-mortem replay.

After All Three Strategies Combined

8-week pilot, aggregate data across 23 production workflows

MONTHLY TOKEN SPEND

\$ 187K → \$ 41K

-78%

Migrated in 8 weeks · zero overtime · zero production incidents

-78%

Monthly Token bill

-41%

P95 response latency

+3.2x

Cache hit rate

0

Incidents from overruns

Turn the strategy into 5 things you can start next Monday

- 1 Tag each Workflow with `endpoint_tag` — tighten default to Tiny
- 2 Use Copilot to extract your 3 longest background docs into JSON Schema
- 3 Wrap LLM clients with the `@token_meter` decorator and budget
- 4 Wire `token_used` / `threshold` / `rollback` metrics into Prometheus
- 5 Weekly cost review: P95 Tokens & unit price by Workflow

Thank You · Thank You